Datasets & Data Pipeline

PhishGuard AI — Multimodal Phishing Detection and Real-Time Browser Protection Platform

B.Tech Final Year Project, A.Y. 2026–27 · Data documentation for project presentation
Every dataset below is **downloadable from a public link or located at a stated path**; every number is measured and reproducible. Synthetic data is flagged everywhere and never mixed with real metrics.

1. Complete dataset inventory (all 3 datasets)

#	Dataset	Origin	Real / Synthetic	Role in the project
A	**PhiUSIIL Phishing URL Dataset** 235,795 URLs × 56 columns	UCI Machine Learning Repository, dataset **#967** (public download, CC BY 4.0)	**REAL — published research data**	**Trains and evaluates our URL machine-learning models** (Random Forest + XGBoost) — the core ML component of the website/URL modality
B	**Real-world sanity set** `url_sanity_realworld.csv` — 40 URLs (20 legit + 20 phishing)	Hand-labelled by our team from live internet sites and active phishing patterns	**REAL — hand-labelled**	**Real-world generalisation check only** (never trained on). Result: 38/40 correct, 0 phishing missed. Exists to catch dataset-style overfitting
C	**Preprocessing demo file** `sample_urls_demo.csv` — 23 rows	Written by our team for this project	**SYNTHETIC — demo only**	**Demonstrates the data-engineering workflow** (upload → inspect → choose operations → execute → results → export). Flagged `is_synthetic=true` in the database; never counted in any accuracy metric

> Runtime scan data (18 rows in the `scans` table) is **live system output**, not a training dataset — it stores every analysis the platform performs with its full evidence trail.

2. Dataset A — PhiUSIIL Phishing URL Dataset (primary training data)

235,795	134,850	100,945	0	54.2 MB
TOTAL URLS	LEGITIMATE (LABEL 1)	PHISHING (LABEL 0)	MISSING VALUES	CSV SIZE

Property	Value
Source / landing page	UCI Machine Learning Repository — dataset 967 (donated 3 Mar 2024)
Download link (page)	https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset
Direct file (zip, 14.7 MB)	https://archive.ics.uci.edu/static/public/967/phiusiil+phishing+url+dataset.zip
Local copy (with checksum)	`datasets/sources/PhiUSIIL_Phishing_URL_Dataset.csv` (+ original zip) — SHA-256 of zip: `0A639FD03AEA6308C5B1C10C92AA23C2CE1505447A9137271865CD0BADC9A59A`
Content	Phishing + legitimate URLs (mostly recent), with 54 engineered features. **We use only the raw `URL` string and the `label`** — features are recomputed by our own code so nothing leaks from the dataset's internals
Label convention	`1` = legitimate, `0` = phishing
License & citation	CC BY 4.0. Cite as: *Prasad, A. & Chandra, S. (2024). PhiUSIIL Phishing URL (Website) [Dataset]. UCI Machine Learning Repository. DOI: 10.1016/j.cose.2023.103545*
Accompanying paper	Prasad & Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework…", *Computers & Security*, 2024

2.1 What we do with it (preprocessing → training), all measured

```
Raw rows ............................ 235,795
Drop conflicting-label URLs ......... 0        (no URL had both labels)
Remove duplicate URLs ............... 425      → 235,370 unique URLs
Feature engineering ................. our own 16 HOST-ONLY features computed
                                      from the URL string (app/ml/url_features.py)
Stratified split, seed = 42 ......... 80% train 188,296 / 20% test 47,074
```

```
                                    (de-duplication BEFORE the split, so an
                                    identical URL can never appear in both)
Train ............................. RandomForest + XGBoost ensemble
Hold-out evaluation (test set only) . RF:  acc 0.956  F1 0.947  ROC-AUC 0.980
                                     XGB: acc 0.958  F1 0.950  ROC-AUC 0.984
Real-world sanity (Dataset B) ....... 38/40 correct, 0 phishing missed
```

Key scientific finding of the project: our first model versions used path-based features and scored ≈99.5% on the hold-out — but that was a **dataset artifact** (every legitimate PhiUSIIL URL is a bare domain, so "has a path" nearly predicted the label). Real-world URLs broke it. Version 3 removed those features, trading ~4 accuracy points for genuine generalisation. This accuracy-vs-generalisation story is the honest highlight of our work.

3. Dataset B — Real-world sanity set (generalisation check)

Property	Value
File	`datasets/url_sanity_realworld.csv` (columns: `url, label, label_note`)
Size	40 URLs — **20 legitimate** (google.com, github.com, stackoverflow.com, python.org, wikipedia.org, …) + **20 phishing patterns** (live impersonation/credential-harvest style URLs)
Origin	Hand-labelled by our team — deliberately in the URL style of the real internet, not of the training dataset
Used for	**Evaluation only — never training, never tuning.** It exists to prove the model generalises beyond PhiUSIIL's URL style
Measured result	**38 / 40 correct; 0 phishing missed.** The 2 misses are benign sites the URL modality alone cannot clear — they are resolved by content fusion in the full pipeline
Stored provenance	Recorded in training metadata (`models/*.metadata.json → realworldSanity`) and shown on the dashboard's Models page

4. Dataset C — Synthetic preprocessing demo file

Property	Value
File	`datasets/sample_urls_demo.csv` — 23 rows × 11 columns (URL features + label)
Origin	**SYNTHETIC** written by our team; contains deliberate imperfections (duplicates, a conflicting-label pair, missing values, an invalid row, a "suspicious" label needing normalisation) so every preprocessing step visibly does something
Used for	The dashboard's **Dataset management → STEP 1–6** workflow demonstration and `scripts/test-api-demo.ps1`
Measured pipeline result	**23 rows in → 17 rows out**: 3 duplicates removed (incl. 1 conflicting-label pair), 2 invalid rows removed, 1 missing value handled, "suspicious → phishing" label mapping, ~31 ms
Database flag	Rows in `datasets` carry `is_synthetic=true` and appear labelled as demo data in the UI — they can never be mistaken for real observations

5. Two preprocessing layers (both reproducible)

5.1 Training-time preprocessing (Dataset A → model)

Implemented in `ai-service/training/train_url_models.py`; every count is persisted to `models/url_models.training.json` (§2.1 box above).

5.2 Interactive workflow preprocessing (Dataset C → audit trail)

Implemented in `ai-service/app/preprocessing/pipeline.py` — a **fixed order, nothing automatic**; the user ticks exactly which operations run:

```
normalize_labels → drop_invalid_rows → convert_types →
detect_conflicting_duplicates → handle_missing → remove_duplicates →
dedupe_on_column → drop/select columns → validate_features (report-only)
```

Every run stores its exact parameters, before/after class balance, per-operation counts, warnings and the full result JSON in the PostgreSQL `preprocessing_runs` table — preprocessing is **auditable and repeatable**.

6. Where the data lives (storage map)

What	Where
Raw real dataset (zip + CSV, checksummed)	`datasets/sources/` in the project
Sanity set & demo CSV	`datasets/url_sanity_realworld.csv`, `datasets/sample_urls_demo.csv`
Trained models (learned from Dataset A)	`models/url_random_forest.joblib`, `models/url_xgboost.joblib` + `*.metadata.json` (provenance, feature list, measured metrics, training timestamp)
Uploaded CSVs at runtime	`uploads/` (originals never modified)
Preprocessed outputs	`uploads/processed/` (downloadable from the dashboard)
PostgreSQL audit tables	`datasets` (with `is_synthetic` flag) · `preprocessing_runs` (parameters + counts + warnings + result JSON) · `model_versions` (registry) · `evaluation_runs` (measured metrics — **NULL if never measured, never a fabricated 0**) · `scans` / `modality_results` / `evidence` (runtime analyses)

Docker deployment	`./models` mounted read-only; uploads on a shared volume — identical layout to local runs

7. How data flows at prediction time (no dataset needed)

```
URL typed by user / browser extension
   → 16 host-only features computed live (same code as training)
   → RF + XGBoost ensemble  → P(phishing)
   → SHAP explains the exact contributors (▲ toward phishing / ▼ toward legitimate)
   → fused with HTML/NLP/form-link evidence → verdict + evidence persisted in PostgreSQL
```

8. Honesty & provenance policies (what makes this defensible)

- **Synthetic data is flagged everywhere** — `is_synthetic` in SQL, a badge in the UI, and a dedicated label in this document.
- **Every metric is measured and stored** (hold-out + sanity + evaluation_runs); anything not measured displays `NULL`, never a fake number.
- **No test-set training** — deduplication happens before the split; the test set is never touched during training.
- **Dataset artifacts are called out, not hidden** — the v1/v2 → v3 feature change is documented with both metric sets.
- **Checksummed raw data** — the PhiUSIIL zip's SHA-256 is recorded in the dataset README, training metadata and this document.

9. References & links (for the presentation)

Primary dataset — PhiUSIIL Phishing URL Dataset
UCI page: https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset
Direct zip: https://archive.ics.uci.edu/static/public/967/phiusiil+phishing+url+dataset.zip · License: CC BY 4.0
Citation: *Prasad, A. & Chandra, S. (2024). PhiUSIIL Phishing URL (Website) [Dataset]. UCI Machine Learning Repository. https://doi.org/10.1016/j.cose.2023.103545*
Paper: *PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning*, Computers & Security, 2024

Local paths to show live during the demo:
Datasets: `Default Project\datasets\` · Models: `Default Project\models\`
Dashboard evidence: `http://localhost:5173/datasets` (STEP 1–6 + dataset table with synthetic flags) · `http://localhost:5173/models` (measured metrics + notes) · `http://localhost:5173/evaluations` (evaluation_runs history)

Generated for the PhishGuard AI B.Tech project (A.Y. 2026–27). All statistics measured from the artifacts in this repository on 15 Oct 2026 — reproducible via `scripts/test-api-demo.ps1` and `ai-service/training/train_url_models.py`.